```
!apt-get update
!apt-get install -y poppler-utils
!pip install -q transformers==4.49.0 jiwer opencv-python-headless google-generativeai pdf2image pydot graphviz
```

```
Get:1 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease [3,632 B]
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Get:3 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64  InRelease [1,581 B]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Hit:6 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:7 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ Packages [87.4 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:9 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64  Packages [2,452 kB]
Get:10 https://cli.github.com/packages stable/main amd64 Packages [357 B]
Get:11 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease [18.1 kB]
Get:12 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [9,880 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Hit:14 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy InRelease
Hit:15 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:16 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,833 kB]
Get:17 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [2,936 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,167 kB]
Get:19 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64 Packages [38.9 kB]
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,303 kB]
Get:21 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6,794 kB]
Get:22 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [7,002 kB]
Get:23 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,615 kB]
Get:24 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [84.0 kB]
Get:25 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [35.6 kB]
Fetched 40.6 MB in 5s (8,100 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  poppler-utils
0 upgraded, 1 newly installed, 0 to remove and 134 not upgraded.
Need to get 186 kB of archives.
After this operation, 697 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 poppler-utils amd64 22.02.0-2ubuntu0.12 [186 kB]
Fetched 186 kB in 1s (215 kB/s)
Selecting previously unselected package poppler-utils.
(Reading database ... 121852 files and directories currently installed.)
Preparing to unpack .../poppler-utils_22.02.0-2ubuntu0.12_amd64.deb ...
Unpacking poppler-utils (22.02.0-2ubuntu0.12) ...
Setting up poppler-utils (22.02.0-2ubuntu0.12) ...
Processing triggers for man-db (2.10.2-1) ...
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 2.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.0/10.0 MB 73.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 566.4/566.4 kB 44.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 101.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.1/3.1 MB 81.5 MB/s eta 0:00:00
```

```
import cv2
import numpy as np
import torch
import re
import string
import unicodedata
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
import jiwer
import google.generativeai as genai
from transformers import AutoProcessor, AutoModelForCausalLM
import os
from pdf2image import convert_from_path
from graphviz import Digraph
from IPython.display import display
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  loader.exec_module(module)
```

```
# VLMs are extremely heavy. If this runs on a CPU, it will freeze or take forever.
if torch.cuda.is_available():
    device = "cuda:0"
```

```
    print("GPU detected")
else:
    device = "cpu"
    print("\nNo GPU detected")

torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32
model = AutoModelForCausalLM.from_pretrained("microsoft/Florence-2-large", torch_dtype=torch_dtype, trust_remote
processor = AutoProcessor.from_pretrained("microsoft/Florence-2-large", trust_remote_code=True)
```

```
GPU detected
config.json:        2.44k/? [00:00<00:00, 199kB/s]

configuration_florence2.py:        15.1k/? [00:00<00:00, 1.46MB/s]

A new version of the following files was downloaded from https://huggingface.co/microsoft/Florence-2-large:
- configuration_florence2.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of th

modeling_florence2.py:        127k/? [00:00<00:00, 6.53MB/s]

A new version of the following files was downloaded from https://huggingface.co/microsoft/Florence-2-large:
- modeling_florence2.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of th

model.safetensors: 100%                                                    1.55G/1.55G [00:09<00:00, 180MB/s]

generation_config.json: 100%                                                    51.0/51.0 [00:00<00:00, 6.15kB/s]

preprocessor_config.json: 100%                                                    806/806 [00:00<00:00, 73.8kB/s]

processing_florence2.py:        48.7k/? [00:00<00:00, 3.94MB/s]

A new version of the following files was downloaded from https://huggingface.co/microsoft/Florence-2-large:
- processing_florence2.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of th

tokenizer_config.json: 100%                                                    34.0/34.0 [00:00<00:00, 4.24kB/s]

vocab.json:        1.10M/? [00:00<00:00, 35.6MB/s]

tokenizer.json:        1.36M/? [00:00<00:00, 50.4MB/s]
```

```
def split_two_page_spread(pil_image):
    img = cv2.cvtColor(np.array(pil_image), cv2.COLOR_RGB2BGR)
    h, w = img.shape[:2]

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Binarize image to make text white and paper black
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)
    col_sums = np.sum(binary, axis=0)

    # Restrict the search for the spine 30% of middle
    mid_start, mid_end = int(w * 0.35), int(w * 0.65)

    # Smooth the projection profile
    kernel_size = max(10, int(w * 0.02))  # Scale smoothing
    smoothed = cv2.blur(col_sums.reshape(1, -1).astype(np.float32), (kernel_size, 1))[0]

    spine_x_relative = np.argmin(smoothed[mid_start:mid_end])
    spine_x = mid_start + spine_x_relative
    min_ink = smoothed[spine_x]

    # Density Check
    left_ink = np.sum(col_sums[:mid_start])
    right_ink = np.sum(col_sums[mid_end:])
    avg_left_ink = left_ink / mid_start
    avg_right_ink = right_ink / (w - mid_end)

    if avg_left_ink > max(min_ink * 2, 100) and avg_right_ink > max(min_ink * 2, 100) and (w > h * 0.5):
        print("-> Detected a 2-column layout! Splitting vertically down the gutter.")
        left_page = img[:, :spine_x]
        right_page = img[:, spine_x:]
        return[
            Image.fromarray(cv2.cvtColor(left_page, cv2.COLOR_BGR2RGB)),
            Image.fromarray(cv2.cvtColor(right_page, cv2.COLOR_BGR2RGB))
        ]
    else:
        return [pil_image]
```

```
def dynamic_multi_slice(pil_image, num_slices=5):
    img = cv2.cvtColor(np.array(pil_image), cv2.COLOR_RGB2BGR)
    h, w = img.shape[:2]

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)
```

```
    slice_height = h // num_slices
    chunks =[]
    last_cut = 0

    for i in range(1, num_slices):
        target_cut = i * slice_height
        search_start = max(last_cut + 10, target_cut - 75)
        search_end = min(h, target_cut + 75)

        # Look for gap between paragraphs
        search_region = np.sum(binary[search_start:search_end, :], axis=1)
        if len(search_region) == 0:
            continue

        kernel_size = max(5, int(h * 0.01))
        smoothed_row_sums = cv2.blur(search_region.reshape(-1, 1).astype(np.float32), (1, kernel_size)).flatten(
        actual_cut = search_start + np.argmin(smoothed_row_sums)

        chunks.append(Image.fromarray(cv2.cvtColor(img[last_cut:actual_cut, :], cv2.COLOR_BGR2RGB)))
        last_cut = actual_cut

    chunks.append(Image.fromarray(cv2.cvtColor(img[last_cut:h, :], cv2.COLOR_BGR2RGB)))
    return chunks
```

```
def extract_pure_text(image_chunk):
    inputs = processor(text="<OCR>", images=image_chunk, return_tensors="pt").to(device, torch_dtype)
    generated_ids = model.generate(
        input_ids=inputs["input_ids"], pixel_values=inputs["pixel_values"],
        max_new_tokens=4096, num_beams=3, do_sample=False
    )
    text = processor.batch_decode(generated_ids, skip_special_tokens=True)[0]
    return text.replace("<OCR>", "").strip()
```

```
def llm_correction_page(raw_text, api_key="##########"):
    try:
        genai.configure(api_key=api_key)

        # Dynamically fetch the best available model instead of hardcoding the name
        valid_models =[m.name for m in genai.list_models() if 'generateContent' in m.supported_generation_method
        chosen_model_name = next((m for m in valid_models if 'flash' in m), valid_models[0])

        model = genai.GenerativeModel(
            model_name=chosen_model_name,
            system_instruction="""
            You are an advanced OCR correction engine for 17th-century Spanish text.
            The raw OCR input contains machine-reading errors due to faded ink.

            YOUR TASK: Predict and reconstruct the actual Spanish words hidden beneath the bad OCR.
            HOWEVER, you MUST preserve the author's historical spelling using ONLY these rules:

            1. 'i' is often used instead of 'j' (e.g., 'Iacinto' -> 'Jacinto', 'Iuan' -> 'Juan').
            2. 'u' is often used instead of 'v' or 'b' (e.g., 'Rauasco' -> 'Ravasco', 'auer' -> 'aver', 'auia' -
            3. Nasal vowels (vowels with a tilde/cap) must be expanded to the vowel + 'n' (e.g., 'cuéta' -> 'cue
            4. 'Z' is often used instead of 'c' (e.g., 'vezes' -> 'veces', 'dize' -> 'dice').
            5. A 'q' with an accent (q̃) MUST be expanded to the full word 'Que' or 'que'.
            6. 'Ç' (cedilla) MUST be converted to 'z' (e.g., 'cobrança' -> 'cobranza').
            7. The long 's' looks like 'f'. Change 'f' to 's' if it forms a dictionary word (e.g., 'efto' -> 'es
            8. CRITICAL ON HYPHENS: DO NOT join hyphenated words at the end of lines. If a word is broken across

            IMPORTANT AND MANDATORY RESTRICTIONS:
            - DO NOT add titles, headers, author names, or metadata (Do not guess the book's name).
            - Output ONLY the cleanly reconstructed text line by line exactly as it appears.
            - DO NOT summarize, fabricate, weave paragraphs together, or restructure the text in any way.
            """,
            generation_config={"temperature": 0.0}
        )
        return model.generate_content(raw_text).text.strip()
    except Exception as e:
        print(f"LLM Error: {e}")
        return raw_text
```

```
def read_ground_truth_safe(file_path):
    """ Reads the ground truth text file safely. Falls back to latin-1 if Spanish accents crash utf-8. """
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            return f.read()
    except UnicodeDecodeError:
        with open(file_path, "r", encoding="latin-1") as f:
            return f.read()
```

```
def clean_for_evaluation(text):
    """ Normalizes strings so Jiwer doesn't artificially penalize the Word Error Rate (WER). """
    text = text.lower().replace('ñ', '<ENYE>')
    text = ''.join(c for c in unicodedata.normalize('NFD', text) if unicodedata.category(c) != 'Mn')
    text = text.replace('<ENYE>', 'ñ')

    text = text.replace('—', ' ')
    text = text.replace('–', ' ')
    text = text.translate(str.maketrans('', '', string.punctuation))
    return re.sub(r'\s+', ' ', text).strip()
```

```
def calculate_metrics(final_pdf_text, gt_path):
    if not os.path.exists(gt_path):
        return

    gt_raw = read_ground_truth_safe(gt_path)

    clean_gt = clean_for_evaluation(gt_raw)
    clean_ai = clean_for_evaluation(final_pdf_text)

    cer = jiwer.cer(clean_gt, clean_ai)
    wer = jiwer.wer(clean_gt, clean_ai)
    mer = jiwer.mer(clean_gt, clean_ai)
    wil = jiwer.wil(clean_gt, clean_ai)

    out = jiwer.process_words(clean_gt, clean_ai)

    # 2. Print them beautifully
    print("\n" + "="*45)
    print("      FINAL PIPELINE EVALUATION METRICS")
    print("="*45)
    print(f"Character Error Rate (CER):        {cer:.4f} ({cer*100:.2f}%)")
    print(f"Word Error Rate (WER):             {wer:.4f} ({wer*100:.2f}%)")
    print(f"Match Error Rate (MER):            {mer:.4f} ({mer*100:.2f}%)")
    print(f"Word Information Lost (WIL):       {wil:.4f} ({wil*100:.2f}%)")
    print("-" * 45)
    print(f"Ground Truth Words: {len(clean_gt.split())} | AI Output Words: {len(clean_ai.split())}")
    print(f"Insertions: {out.insertions} | Deletions: {out.deletions} | Substitutions: {out.substitutions}")
    print("="*45 + "\n")
```

```
import PIL.Image
PIL.Image.MAX_IMAGE_PIXELS = None
if __name__ == "__main__":
    PDF_PATH = "Buendia.pdf"
    GT_TXT_PATH = "Buendia.txt"
    GEMINI_API_KEY = "######"

    print(f"\nProcessing PDF: {PDF_PATH}")
    pdf_pages = convert_from_path(PDF_PATH)
    print(f"Found {len(pdf_pages)} pages in document.")

    final_document_text = ""

    for i, pdf_page in enumerate(pdf_pages):
        print(f"\nProcessing page {i+1} / {len(pdf_pages)}...")
        page_raw_text = ""

        # Detect and handle 2-page spreads
        sub_pages = split_two_page_spread(pdf_page)

        # Extract text
        for sub_page in sub_pages:
            chunks = dynamic_multi_slice(sub_page, num_slices=5)
            for chunk_idx, chunk in enumerate(chunks):
                page_raw_text += extract_pure_text(chunk) + "\n"
        page_corrected_text = llm_correction_page(page_raw_text, api_key=GEMINI_API_KEY)

        final_document_text += page_corrected_text + "\n"

        # Separate pages with a delimiter to match the ground truth format
        if i < len(pdf_pages) - 1:
            final_document_text += "\n———————————————————\n\n"

    print("\nText extraction and correction complete.")

    with open("final_pdf_text.txt", "w", encoding="utf-8") as f:
        f.write(final_document_text)
    print("Output saved to 'final_pdf_text.txt'.")

    calculate_metrics(final_document_text, gt_path=GT_TXT_PATH)
```

```
Processing PDF: Buendia.pdf
Found 3 pages in document.

Processing page 1 / 3...
-> Detected a 2-column layout! Splitting vertically down the gutter.

Processing page 2 / 3...
-> Detected a 2-column layout! Splitting vertically down the gutter.

Processing page 3 / 3...
-> Detected a 2-column layout! Splitting vertically down the gutter.

Text extraction and correction complete.
Output saved to 'final_pdf_text.txt'.

============================================
      FINAL PIPELINE EVALUATION METRICS
============================================
Character Error Rate (CER):        0.0556 (5.56%)
Word Error Rate (WER):             0.1513 (15.13%)
Match Error Rate (MER):            0.1474 (14.74%)
Word Information Lost (WIL):       0.2504 (25.04%)
--------------------------------------------
Ground Truth Words: 608 | AI Output Words: 621
Insertions: 16 | Deletions: 3 | Substitutions: 73
============================================
```